PoseidonOS: a resource governor for a software-defined vessel

Team 6, Maritime Dual-Use Hackathon 2026, Fincantieri challenge. PoseidonOS (Python package `shipos`) runs on the ship's fixed compute rack, between platform automation and the mission systems. It owns eight finite budgets (power, heat rejection, deck, mast, mass/stability, data links, compute, response time) and decides which guest equipment may come aboard, where it goes, who wins a contention, and what is shed when a generator trips. It runs offline and is deterministic. There is no ML in the decision path, and every decision carries one sentence generated from the constraint that bound it. Civil use only.

Run it: one command

```
make install   # once, online: uv sync --locked (Python 3.11+, uv)
make demo      # offline: dashboard at http://127.0.0.1:8000/
```

On Windows without `make`, use `.\tools\demo.ps1 -Prepare` once, then `.\tools\demo.ps1`. The decision engine is never mocked: the dashboard, the simulator and the tests all call the same `shipos.engine` functions. `make check` runs lint, the no-magic-numbers guardrail and the full test suite (491 passed at submission).

Architecture

```
data/ship  (Annex A)      data/guests  G1–G7 cards       data/priorities  (swappable)
data/assumptions (D6)     schema/guest_manifest.json      data/envelopes   (A.10)
           │                         │                              │
           └──────────── frozen contracts (shipos/contracts.py) ────┘
                                     │
sim: seeded 1 Hz Runner ── event timeline (battery, genset trip, card, priority swap, crew)
  │  WorldSnapshot                   ▲  Decisions
  ▼                                  │
physics: 8 budgets, power→heat 1:1, cooling(T_sea), reserve(speed, gensets), mass×position
  │
engine kernel: worst coincident draw over modes × exclusions × ship-state conflicts
  ├─ D1 admission – D2 placement – D3 arbitration – D4 shedding ── rationale.render()
  ▼
crew policy (14 / 6 / 0) ── hash-chained JSONL log (replayable) ── FastAPI + WebSocket
                                                                    └ one screen: schematic,
                                                                      8 gauges, decision log
```

Decision classes: component and declared objective

	Component	What it optimises
Manifest	`shipos/contracts.py` `GuestManifest`, `schema/guest_manifest.schema.json`, `data/guests/*.yaml`	Expresses what Kubernetes cannot: modes (min/nominal/peak, duration, next mode, interruptible), heat path (water/air), discrete positions (preferred/allowed), mass per part, ship-state conditions (speed, course, noise, deck, emission, wind, sea state), part-of / requires / excludes (absolute or conditional), safety-critical, capability by mission. Guide: `docs/ADDING_A_CARD.md`.
D1 Admission ≤ 30 s	`shipos/engine/admission.py`	Admit only if every mission-envelope corner holds (slow and cruise bands × one genset down × 30 °C water), never pier-side. Otherwise rank fixes by priority-weighted capability lost (permanent first, then contingent throttle / standby / interlock), then by actions touched. *yes-if* only if the fix costs less than the card returns. Names the binding class and states whether it holds with one genset down. At sea it answers *not now* with the next window. At 0 crew it proposes logical re-tasking.
D2 Placement	`shipos/engine/placement.py`	Lexicographic: feasible in every corner, then fewest moves, then largest minimum normalised margin (stability, mission lines), then most future flexibility (catalogue cards still mountable), then preferred positions, then capability after placement effects. Exhaustive search; rejected alternatives are reported with a reason.
D3 Right of way ≤ 2 s	`shipos/engine/arbitration.py`	Hard constraints first (drone below 20 % battery must land within 12 min). Then minimise priority-weighted capability-

	Component	What it optimises
		minutes lost (the loser's re-run and wait, plus the winner's blocked follow-on). The conflict is found by intersecting the two guests' conditions on ship state; no guest pair is hard-coded.
D4 Degradation, first action ≤ 5 s, full sequence ≤ 60 s	`shipos/engine/shedding.py`	Feasible at every instant of a forecast horizon. Order: fewest mission-critical guests touched, then lowest tier (defer / throttle to low-power, then standby, then off), then least capability-minutes, then fewest actions. Never touches a safety-critical guest, the protected D3 winner or a non-interruptible mode. If nothing needs shedding, it still states every guest's loss and the armed shed ladder.
D5 Crew as a setting	`shipos/crew/policy.py`, `lifecycle.py`	Rule: "flight deck clear and confirmed before recovery". At 14 or 6 crew a human confirms. At 0 it is automatic only on fresh deck evidence, and impossible if the deck is occupied. Memo: `deliverables/03_scenario_C_memo/`.
D6 Physical Account	`data/assumptions/*.yaml`, `tools/physical_account.py`	Every invented number has a value, unit, range, reason and source, and is read through `A("<id>")`. CI fails on a numeric literal in physics or engine code, and the registry lists every use site.

Shared by D1–D4: `engine/kernel.py` computes the **worst coincident draw**, the highest power or heat combination of modes the guests could actually be in at once, given exclusions, interlocks and ship-state conflicts. `engine/rationale.py` builds each sentence from the binding `ConstraintResult` (resource, capacity, demand, corner, contributors). There are no canned strings.

What it does on the reference scenarios

The full log is in `deliverables/05_validation/` (54/54 checks, real engine).

- **A** (G1, G2, G5 installed; jury card G4): *yes-if* in about 50 ms. "At slow speed in 30 °C water with one genset down, guests need 101.5 kW against 100 kW usable … Admit G4 if G2's winch waits while G4 charging." Five ranked alternatives (throttle G5, shed G1, …). Change G4 charging to 70 kW and the fix becomes *G5 to low-power (–37 kW)*. Change its mass to 20 t and it is refused on hangar mass.
- **A′**: a port-only card at sea gives *not now: next port call within 14 days*. At 0 crew the same answer also re-tasks G5's edge compute and 92 free rack cores.
- **B min 8**: G4 wins under both priority files (hard 12-min deadline vs 41 min to the end of the leg). Survey primary: *G3 holds 15 min, cost 23 survey-min*. SAR primary: *G3 suspends, 35 kW is reserved for G4, cost 83 survey-min*.
- **B min 12**: with G4 charging over G3's non-interruptible recharge (106.5 kW against 100 kW), survey primary throttles G5 to its 8 kW low-power mode at t+1 s, and SAR primary defers G3's charge 5 min and keeps G5. Each sentence states every guest's loss. On the baseline timeline the drone is still airborne at minute 12 (49.5 kW drawn), so the honest answer is "no shedding needed" plus the armed ladder.
- **Determinism**: same seed and timeline give the same SHA-256 log hash. Export and *Verify by replaying* re-executes the engine byte for byte.

Our cards: G6 Towed passive acoustic array (and G7)

`data/guests/G6.yaml` is the team's original card: a civil towed hydrophone array for marine-mammal and ambient-noise monitoring. It is a 2.5 t drum-and-winch module in a Bay 1 half or Bay 2. It draws 15 kW while deploying or recovering over the stern ramp and 4 kW while listening. Listening needs 4–6 kn, a steady course, quiet, and no deck machinery. It excludes G2's winch, and a transect can be paused for 30 min without being re-run. All 17 numbers are invented assumptions (`data/assumptions/g6.yaml`). In Scenario A it is admitted onto the free Bay 1 half. With G2, G3 and G4 aboard it is refused ("bay1 holds 2 units but G3 and G6 need 3"). At sea the answer is *not now: port only, 3 people, 4 h*.

`data/guests/G7.yaml` is a second invented card that exercises the B.0 fields no other card sets: a survivor-care medical module that is **safety-critical**, **requires G5** (telemedicine over SATCOM) and has a 10 kW compressor-start peak. Without G5 it is refused ("G7 requires G5 …"), and D4 never sheds it ("G7 keeps treating").

Limitations

- Stability is a weighted-tonnes proxy (Annex A.4 multipliers), not hydrostatics. Sea-state and weather mapping, and A.6 transition times, are engineering upper bounds, not ship trials.

- Cooling inertia (+30 kW for 60 s) is modelled in physics but not used by the D4 planner. The A.11 250 kg portable limit is declared but not yet enforced, because no card installs at sea.
- Zero-crew deck evidence is simulated (no camera integration). Sessions live in memory: export before restarting. Timing ceilings are measured in tests on a laptop, not on the A.7 rack.
- Our reading of the brief differs in one place: Annex B says "any four exceed the 10 t margin", but G1+G2+G4+G5 (7.61 t) and G1+G3+G4+G5 (9.61 t) fit. We follow the card arithmetic.

With another week

1. A MIP formulation (OR-Tools) of D1/D2 as a cross-check on the exhaustive search, for larger catalogues.
2. Use the cooling-inertia buffer in D4, so a short overlap rides through instead of shedding.
3. Battery *forecasting* (not deciding) from drone telemetry, to start the 6-crew deck clearance 15 min early instead of reacting to the 20 % alert.
4. At-sea installation windows for portable modules (≤ 250 kg) on the live Copernicus feed.
5. A second implemented crew rule ("no winch while the survey vehicle listens") at 6 and 0 crew.

Deliverables index: `deliverables/README.md`. Plan and arithmetic: `docs/PLAN.md`.